



华南师范大学

本科学生实验（实践）报告

院 系：计算机学院

实验课程：编译原理实验

实验项目：实验 5: TINY 扩充语言的中间代码生成

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 1 学期

专 业：计算机科学与技术

班 级：计科 2 班

学生姓名：袁知本

华南师范大学教务处

华南师范大学实验报告

学生姓名 袁知本 学 号 20232131006
专 业 计算机科学与技术 年级、班级 2023 级 计科 2 班
课程名称 编译原理实验 实验项目 实验 5: TINY 扩充语言的中间代码生成
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2025 年 6 月 12 日
实验指导老师 黄煜廉 实验评分 _____

一、实验内容

必做内容:

(一) 为 Tiny 语言扩充的语法有

1. 增加 while 循环;
2. 增加 for 循环(书写格式类似于 C 语言的 for 语言格式)
3. 扩充算术表达式的运算符: ++ (前置自增 1)、-- (前置自减 1) 运算符(类似于 C 语言的++和--运算符,但不需要完成后置的自增 1 和自减 1)、求余%、乘方^;
4. 扩充扩充比较运算符: <=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等比较运算符;

(二) 对应的语法规则

1. while 循环语句的语法规则: while-stmt-->while exp do stmt-sequence enddo
2. for 语句的语法规则(类似于 C 语言的 for 语言格式):
书写格式: for(循环变量赋初值;条件;循环变量自增或自减 1) 语句序列
3. ++ (前置自增 1)、-- (前置自减 1) 运算符、求余%、乘方^等运算符的文法规则请自行组织
4. <=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等运算符的文法规则请自行组织
5. TINY 语言的 BNF 语法规则如下:

```

program->stmt-sequence

stmt-sequence-> stmt-sequence ; statement |statement
statement->if-stmt      |repeat-stmt      |assign-stmt      |
read-stmt|write-stmt

if-stmt->if exp then stmt-sequence end
| if exp then stmt-sequence else stmt-sequence end

repeat-stmt->repeat stmt-sequence until exp

assign-stmt->identifier := exp

read-stmt->read identifier

write-stmt->write exp

exp->simple-exp comparison-op simple-exp |simple-exp
comparison-exp-> < | =

simple-exp->simple-exp addop term | term
addop-> + |-

term->term mulop factor | factor
mulop-> * | /

factor->(exp) | number | identifier

```

6. 为 tiny 语言增加一种新的语句, ID::=正则表达式 (同时增加正则表达式的赋值运算符号::=)
7. 为 tiny 语言增加一个符合上述 for 循环的书写格式的文法规则
8. 为 tiny 语言增加一个符合上述 while 循环的书写格式的文法规则
9. 为了实现以上的扩充或改写功能, 还需要注意对原 tiny 语言的文法规则做一些相应的改造处理

(三) 实验要求

- (1) 要提供一个源程序编辑的界面, 以让用户输入源程序 (可输入, 可保存、可打开源程序)

- (2) 要提供一个查看所生成的四元组中间代码的界面
- (3) 实验 3 的实现只能选用的程序设计语言为: C++
- (4) 要求应用程序的操作界面应为 Windows 界面
- (5) 输入: 一个扩充语法的 TINY 语言源程序; 输出: 输出所生成的四元组的中间代码

选做内容:

书写完善的软件文档

二、实验目的

1. 实现一个带图形化界面的 TINY 扩充语言的四元组中间代码生成程序
2. 用户可以通过直接输入、拖拽文件、通过文件对话框等方式输入 TINY 源程序, 可由用户选择生成四元组中间代码
3. 深入了解语法树生成的基本原理和实现方法, 同时掌握通过 QT 开发图形化界面项目的要点
4. 深入了解 TINY 语言的文法规则, 能够在理解原有文法的基础上进行文法的改造和添加处理

三、实验文档:

注: 本次实验中的必做内容均已完成, 同时也完成了全部的选做内容

1. 实验需求分析

本实验的目标是用 C++ 语言实现一个具有图形化界面的 TINY 扩充语言的语法树生成系统, 要为用户实现文件拖拽、文件对话框选择和直接输入等多种输入 TINY 源代码的方式, 需实现由用户选择生成语法树的功能和源代码文件保存功能, 并且在生成语法树时要有源码匹配结果状态的显示, 同时, 通过 QT 平台构建一个用户友好型可视化图形界面

功能需求

1) 用户界面设计

用户界面包括状态信息栏、TINY 源代码输入区、四元组中间

代码区共 3 个模块：

- 状态信息栏：显示提示信息，显示用户输入的 TINY 源代码的语法规则匹配结果；
- TINY 源代码输入区：可以通过直接输入 TINY 源代码、拖拽文件和打开文件对话框等三个方式将 TINY 源代码输入到输入区中，点击【保存源程序】可以进行保存，点击【读取源程序文件】可以实现打开文件对话框选择文件
- 四元组中间代码生成区：借助 Qt 中的 Plain Text Edit 控件，实现生成四元组中间代码后的输出展示，该功能由用户点击按钮【生成四元组中间代码】实现

2) TINY 语言文法规则的扩充与改写

本次实验的程序基于原有的 TINY 语言文法规则，在其基础上做了部分修改和语句扩充，具体如下：

1. for 语句、while 语句的新增与改写（C 语言风格）；
2. 运算符（自增++、自减--、求余%、乘方^等）、比较符号（小于等于<=、大于>、不等于<>、大于等于>=等）的新增；

3) 生成四元组中间代码

基于原有的 TINY 语言文法规则，加上扩充的内容以及改写的部分，通过 match 函数实现读取 token 后分类，生成四元组中间代码，将其通过 Qt 中的 Plain Text Edit 控件，实现生成四元组中间代码后的输出展示

2. 实验环境

硬件设备：基于 x64 的处理器，RAM16.0GB

软件设备：Windows11 家庭版，

Qt5.14.2，

Qt Creator 4.11.1(Community)

3. 系统设计

1) 简洁方便的交互界面:

用 Qt 开发的可视化交互界面如下图所示:

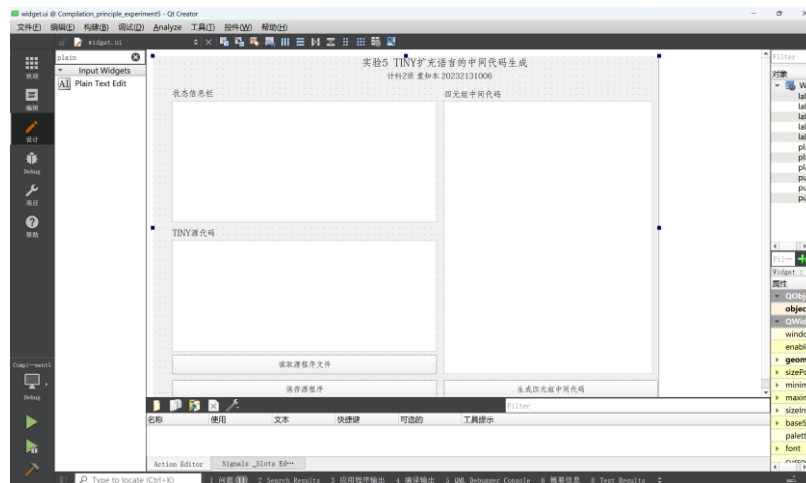


图 3-1-1 交互界面

2) TINY 语言语法规则的扩充与改写:

本功能包含以下部分:

- **for 语句文法改写:**

根据实验三要求, 把 TINY 语言原有的 for 语句的文法规则修改为如下:

`for_stmt -> for(assign_stmt;exp,assign_stmt) stmt_sequence`

由于要修改为 C 风格的 for 语句, 故原有的 for 语句文法规则中的 `downto`、`to` 都将弃用

- **while 语句文法扩充:**

在原本的 TINY 语言语法规则中, 不存在 while 语句, 故我们需要扩充该语句的文法到 TINY 语言中:

`while_stmt → while exp do stmt_sequence enddo`

通过 `do` 判断 while 语句中循环体的开始, `enddo` 作为来判断 while 语句的结束

- **运算符的新增:** 本次实验中新增了++ (前置自增 1)、--

(前置自减 1) 运算符号、求余%、乘方^等运算符号，其中实验要求的++和--运算我都做了额外的后置版本扩充，使得扩充后的 TINY 语言功能更加强大，而根据 C++中运算符的优先级顺序，我们可知 TINY 语言中运算符的优先级顺序（从上到下为由高到低）如下：

1. ++ --
2. ^
3. * / %
4. + -
5. +=

因此，根据上述的优先级顺序，我们可以制定文法规则，转换成 EBNF 后如下：

plusplus -> ++

subsub -> --

addop -> + | -

power-> factor {powop factor}

powop -> ^

factor -> (exp) | number | identifier

mulop -> * | / | %

- **比较运算符新增：**本次实验中新增了<=(小于等于)、>(大于)、>=(大于等于)、<>(不等于)等比较运算符号，与运算符同理，我们也可以写出其语法规则，转 EBNF 后如下：

andop -> and

andexp -> simple-exp [comparison-op simple-exp]

orop -> or

orex -> andexp {andop andexp}

exp -> orexp {orop orexp}

notop-> not

```
notexp -> {notop} power  
simple-exp -> simple-exp addop term | term  
term -> notexp {mulop notexp}
```

3) 生成四元组中间代码:

基于原有的 TINY 语言语法规则, 加上扩充的内容以及改写的部分, 通过 match 函数实现读取 token 后分类, 生成四元组中间代码, 将其通过 Qt 中的 Plain Text Edit 控件输出显示, 实现生成四元组中间代码后的输出展示

4. 系统实现

4.1 用户界面设计

4.1.1 布局:

主窗口包含状态信息栏、TINY 源代码输入栏和语法分析显示窗口, TINY 源代码输入框支持拖拽文件、通过文件对话框选取以及直接输入三种方式导入 TINY 源代码; 状态信息栏显示通过语法规则匹配 TINY 源代码的结果; 语法分析显示窗口能够显示生成的四元组中间代码; 按钮【读取 TINY 源程序】实现通过文件对话框选取 TINY 源代码文件; 按钮【保存源程序】实现将当前输入在输入框中的 TINY 源程序保存下来; 按钮【生成四元组中间代码】实现借助 Qt 中的 Plain Text Edit 控件输出显示四元组中间代码
主窗口界面如下图所示:



图 4-1-1 主窗口控件布局

4.1.2 交互逻辑：

点击按钮【读取源程序文件】，即可打开文件对话框，在文件对话框中选择你要导入的 TINY 源代码文件，单击【确定】即可导入示例如下：

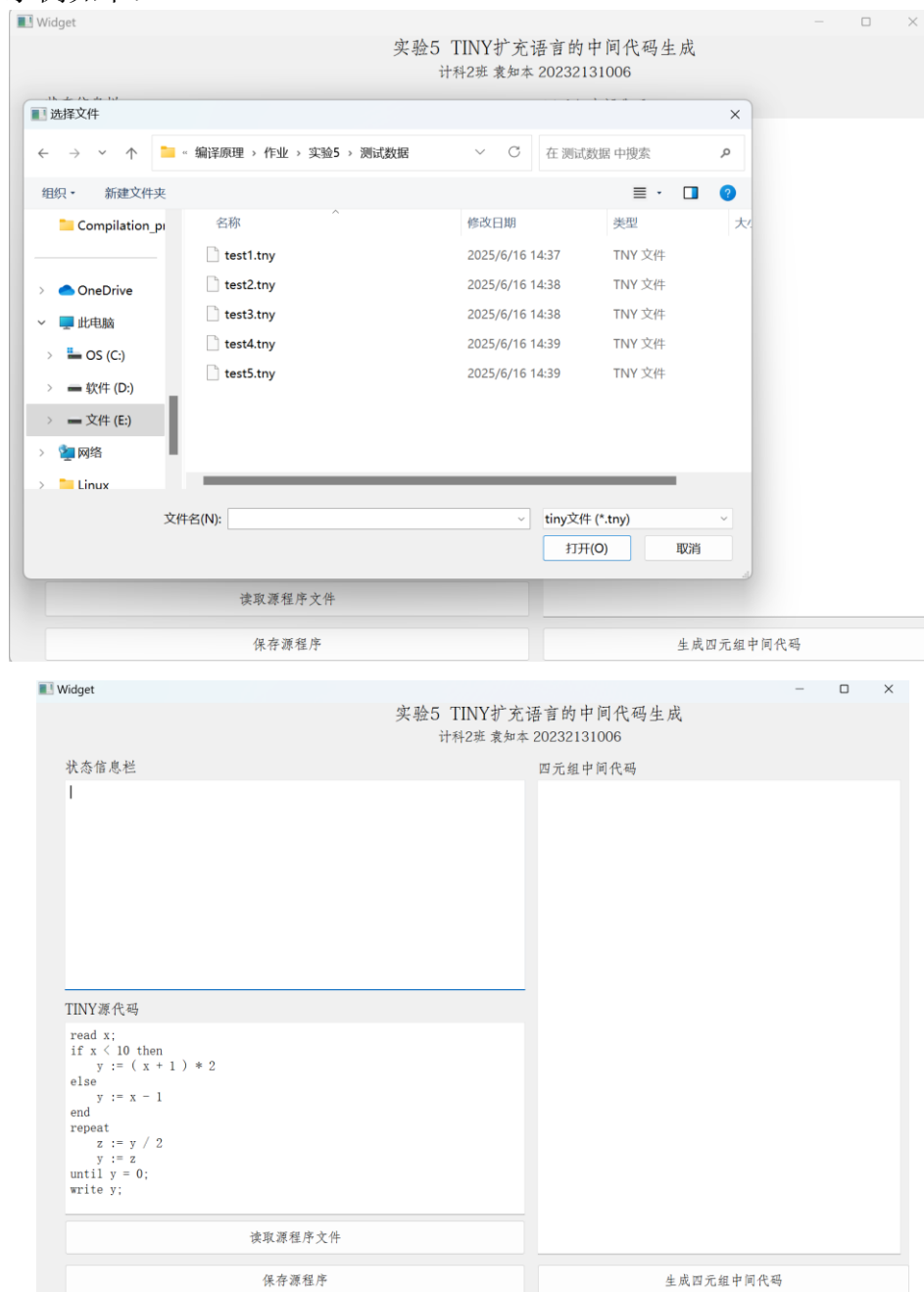


图 4-1-2、图 4-1-3 TINY 源代码导入功能展示

如上图所示，除了直接输入、文件拖拽的方式导入 TINY 源代码，也可以通过点击【读取源程序文件】实现通过打开文件对话框，选择本地的 TINY 源程序导入

在导入或手动输入了 TINY 源代码后，点击按钮【保存源程序】，即可打开文件对话框，自定义保存的文件名、保存的位置路径，点击【保

存】即可将当前输入到 TINY 源代码输入框中的 TINY 源代码保存到
你刚才指定的文件和位置中，其展示分别如下：

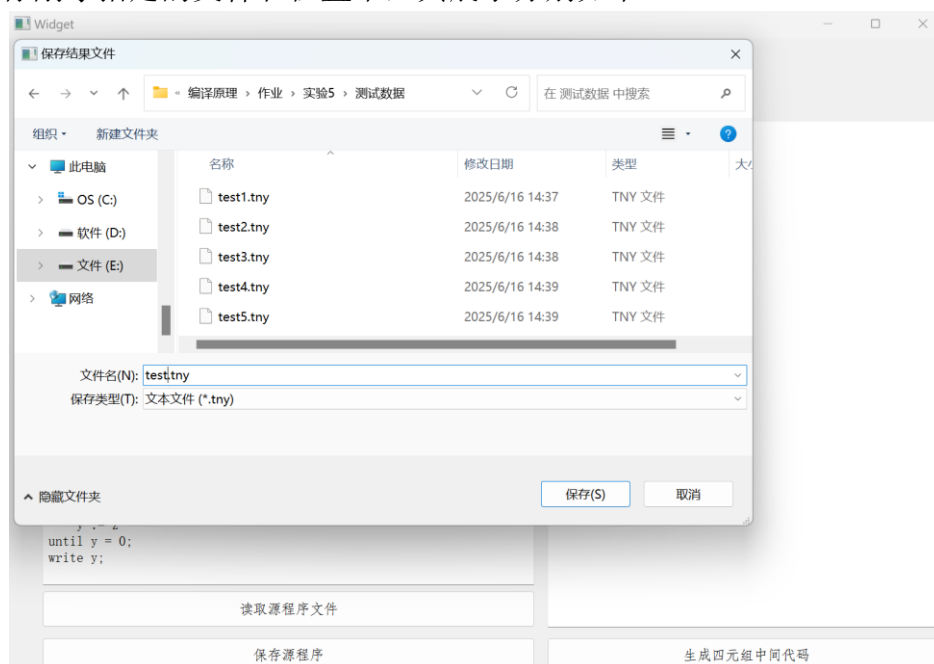


图 4-1-4 保存源代码功能演示（打开文件对话框）



图 4-1-5 保存源代码功能演示（保存成功的提示）

在导入或手动输入了 TINY 源代码后，点击按钮【生成四元组中间代码】，系统将会将这段 TINY 源代码写入到一个临时文件 tempfile 中，然后开始根据修改与扩充后的 TINY 语言语法规则，对输入的 TINY 源代码进行匹配，并生成对应的四元组中间代码，完成后将四元组借助 Qt 中的 Plain Text Edit 控件输出显示，同时，状态信息栏中也会显示匹配的结果，运行效果如下：



图 4-1-6 语法树构建与展示功能演示

4. 1. 3 用户交互界面的实现:

控件的实现:

如图 4-1-1 所示, 用户交互界面的有按钮和文本编辑框 (Plain Text Edit) 等若干控件, 均可通过 Qt 的工具栏拖拽进行添加, 然后给文本编辑框等控件设置对应的变量, 对应的按钮设置响应函数, 其代码如下图所示:

```

1  #ifndef WIDGET_H
2  #define WIDGET_H
3
4  #include <QWidget>
5
6  QT_BEGIN_NAMESPACE
7  namespace Ui { class Widget; }
8  QT_END_NAMESPACE
9
10 ▼ class Widget : public QWidget
11 {
12     Q_OBJECT
13
14 public:
15     Widget(QWidget *parent = nullptr);
16     ~Widget();
17
18 private slots:
19     // 读取源程序的按钮控件
20     void on_pushButton_Load_clicked();
21     // 保存源程序的按钮控件
22     void on_pushButton_Save_clicked();
23     // 生成四元组中间代码的按钮控件
24     void on_pushButton_Generate_clicked();
25
26 private:
27     Ui::Widget *ui;
28 };
29 #endif // WIDGET_H

```

图 4-1-7 响应函数一览

```

21 ▼ class Ui_Widget
22 {
23     public:
24         QLabel *label_2;
25         QLabel *label_1;|
26         QLabel *label_3;
27         QPlainTextEdit *plainTextEdit_Status;
28         QPlainTextEdit *plainTextEdit_Tiny;
29         QLabel *label_4;
30         QPushButton *pushButton_Load;
31         QPushButton *pushButton_Save;
32         QPushButton *pushButton_Generate;
33         QLabel *label_5;
34         QPlainTextEdit *plainTextEdit_Generate;

```

图 4-1-8 控件变量一览

4.2 功能实现

4-2-1 for、while 语句文法规则匹配实现：

根据实验三要求，把 TINY 语言原有的 for 语句的文法规则修改为如下：

for_stmt -> for(assign_stmt;exp;assign_stmt) stmt_sequence

据此，实现 for 语句文法规则匹配的代码如下图所示：

```

499 // 修改后的for_stmt
500 // for_stmt -> for(assign_stmt;exp;assign_stmt) stmt_sequence
501 ▼ TreeNode* for_stmt(void)
502 {
503     TreeNode* t = newStmtNode(Fork);
504     match(FOR);
505     match(LPAREN);
506     // 处理初始化赋值部分
507     if (t) t->child[0] = assign_stmt();
508     match(SEMI);
509     // 处理条件表达式部分
510     if (t) t->child[1] = exp();
511     match(SEMI);
512     // 处理更新赋值部分
513 ▼ if (t)
514     {
515         if (token == PLUSPLUS) t->child[2] = plusplus_stmt();
516         else if (token == SUBSUB) t->child[2] = subsub_stmt();
517         else t->child[2] = assign_stmt();
518     }
519     match(RPAREN);
520     match(LPM);
521     if (t) t->child[3] = stmt_sequence();
522     match(RPM);
523     return t;
524 }

```

图 4-2-1 for 语句文法规则匹配函数

在原本的 TINY 语言文法规则中，不存在 while 语句，故我们需要扩充该语句的文法到 TINY 语言中：

while_stmt $\rightarrow$ while exp do stmt_sequence enddo

通过 do 判断循环体的开始，enddo 作为来判断 while 语句的结束
实现 while 语句文法规则匹配的代码如下图所示：

```

526 // 新增while_stmt
527 // 实验三: while_stmt -> while(exp) stmt_sequence endwhile
528 // 实验五: while_stmt -> while exp do stmt_sequence enddo
529 ▼ TreeNode* while_stmt(void)
530 {
531     // 建立语法树结点: WhileK 表示 “while” 语句
532     TreeNode* t = newStmtNode(WhileK);
533
534     // 关键字 while
535     match(WHILE);
536
537     // 条件表达式 exp
538     // child[0] 存放循环条件
539     if (t) t->child[0] = exp();
540
541     // 关键字 do
542     match(DO);
543
544     // 循环体 stmt_sequence
545     // child[1] 存放循环体
546     if (t) t->child[1] = stmt_sequence();
547
548     // 关键字 enddo
549     match(ENDDO);
550
551     return t;
552 }

```

图 4-2-2 while 语句文法规则匹配函数

4-2-2 新增运算符和比较符号的匹配实现:

根据之前分析出来的新增的运算符和比较符号文法，我们可以给出如下的文法规则匹配函数：

```

195         case '*':
196             currentToken = TIMES;
197             break;
198         case '/':
199             currentToken = OVER;
200             break;
201         case '(':
202             currentToken = LPAREN;
203             break;
204         case ')':
205             currentToken = RPAREN;
206             break;
207         case '[':
208             currentToken = LPM;
209             break;
210         case ']':
211             currentToken = RPM;
212             break;
213         case ';':
214             currentToken = SEMI;
215             break;
216         case '%':
217             currentToken = MOD;
218             break;
219         case '^':
220             currentToken = POWER;
221             break;

```

图 4-2-3 新增运算符匹配

```

167
168
169 case '+':
170     c = getNextChar(); // Read the next character
171     if (c == '=')
172     {
173         currentToken = PLUSEQ; // Token for +=
174     }
175     else if (c == '+')
176     {
177         currentToken = PLUSPLUS; // Token for ++
178     }
179     else
180     {
181         ungetNextChar(); // Backup if it's not += and ++
182         currentToken = PLUS; // Token for +
183     }
184     break;
185 case '-':
186     c = getNextChar(); // Read the next character
187     if (c == '-')
188     {
189         currentToken = SUBSUB; // Token for --
190     }
191     else
192     {
193         ungetNextChar(); // Backup if it's not --
194         currentToken = MINUS; // Token for -
195     }
196     break;

```

图 4-2-4 新增多元运算符匹配

```

128         state = DONE;
129         switch (c)
130         {
131             case EOF:
132                 save = FALSE;
133                 currentToken = ENDFILE;
134                 break;
135             case '=':
136                 currentToken = EQ;
137                 break;
138             case '>':
139                 c = getNextChar(); // Read the next character
140                 if (c == '=')
141                 {
142                     currentToken = RTEQ; // Token for >=
143                 }
144             else
145             {
146                 ungetNextChar(); // Backup if it's not >= or <>
147                 currentToken = RT; // Token for >
148             }
149             break;
150             case '<':
151                 c = getNextChar(); // Read the next character
152                 if (c == '=')
153                 {
154                     currentToken = LTEQ; // Token for <=
155                 }
156                 else if (c == '>')
157                 {
158                     currentToken = NOTEQ; // Token for <>
159                 }
160                 else
161                 {
162                     ungetNextChar(); // Backup if it's not <=
163                     currentToken = LT; // Token for <
164                 }
165             }
166             break;

```

图 4-2-5 新增多元比较符号匹配

```

427 // 新增乘方运算
428 TreeNode* power(void)
429 {
430     TreeNode* t = factor();
431     while ((token == POWER))
432     {
433         TreeNode* p = newExpNode(OpK);
434         if (p != NULL) {
435             p->child[0] = t;
436             p->attr.op = token;
437             t = p;
438             match(token);
439             p->child[1] = factor();
440         }
441     }
442     return t;
443 }

```

图 4-2-6 新增乘方运算符匹配

```

181 // 新增判断前置自增的文法 (plusplus)
182 TreeNode* plusplus_stmt(void)
183 {
184     // 创建一个赋值语句节点，类型为 AssignK
185     TreeNode* t = newStmtNode(AssignK);
186     if ((t) && (token == PLUSPLUS)) {
187         // 保存变量名
188         match(PLUSPLUS); // 消耗掉前置 ++ 符号
189         if (t && token == ID) {
190             // 记录自增操作的目标变量名
191             t->attr.name = copyString(tokenString);
192             // 构造表达式部分：等价于 x = x + 1
193             TreeNode* p = newExpNode(OpK);
194             if (p != NULL) {
195                 // 左子节点：标识符节点，表示原始变量 x
196                 TreeNode* left = newExpNode(IdK);
197                 if (left) {
198                     left->attr.name = copyString(t->attr.name);
199                 }
200                 p->child[0] = left;
201
202                 // 操作符：PLUS
203                 p->attr.op = PLUS;
204
205                 // 右子节点：常量节点，表示字面量 1
206                 TreeNode* right = newExpNode(ConstK);
207                 if (right) {
208                     right->attr.val = 1;
209                 }
210                 p->child[1] = right;
211
212                 // 将整个加法表达式作为赋值语句的右子节点
213                 t->child[0] = p;
214             }
215         }
216         // 消耗掉标识符 token
217         match(ID);
218     }
219     return t;
220 }

```

图 4-2-7 新增自增符号运算文法规则匹配函数（同时实现了前置和后置自增）

```

140 // 新增判断前置自减的文法(subsub)
141 TreeNode* subsub_stmt(void)
142 {
143     // 创建一个赋值语句节点, 类型为 AssignK
144     TreeNode* t = newStmtNode(AssignK);
145     if ((t) && (token == SUBSUB)) {
146         // 保存变量名
147         match(SUBSUB); // 消耗掉前置 -- 符号
148         if (t && token == ID) {
149             // 记录自增操作的目标变量名
150             t->attr.name = copyString(tokenString);
151             // 构造表达式部分: 等价于 x = x - 1
152             TreeNode* p = newExpNode(OpK);
153             if (p != NULL) {
154                 // 左子节点: 标识符节点, 表示原始变量 x
155                 TreeNode* left = newExpNode(IdK);
156                 if (left) {
157                     left->attr.name = copyString(t->attr.name);
158                 }
159                 p->child[0] = left;
160
161                 // 操作符: MINUS
162                 p->attr.op = MINUS;
163
164                 // 右子节点: 常量节点, 表示字面量 1
165                 TreeNode* right = newExpNode(ConstK);
166                 if (right) {
167                     right->attr.val = 1;
168                 }
169                 p->child[1] = right;
170
171                 // 将整个加法表达式作为赋值语句的右子节点
172                 t->child[0] = p;
173             }
174         }
175         // 消耗掉标识符 token
176         match(ID);
177     }
178     return t;
179 }

```

图 4-2-8 新增自减符号运算文法规则匹配函数（同时实现了前置和后置自减）

```

82 //P394
83 //lineno: 961
84 TreeNode* statement(void)
85 {
86     TreeNode* t = NULL;
87     switch (token) {
88         case IF: t = if_stmt(); break;
89         case REPEAT: t = repeat_stmt(); break;
90         case ID: t = assign_stmt(); break;
91         case READ: t = read_stmt(); break;
92         case WRITE: t = write_stmt(); break;
93         case FOR: t = for_stmt(); break;
94         case WHILE: t = while_stmt(); break;
95         case SUBSUB: t = subsub_stmt(); break;
96         case PLUSPLUS: t = plusplus_stmt(); break;
97         default: syntaxError("unexpected token -> ");
98                 printToken(token, tokenString);
99                 token = getToken();
100                 break;
101     } /* end case */
102     return t;
103 }

```

图 4-2-9 修改后的 statement 函数

对于上面的 statement 函数，case 中 SUBSUB 和 PLUSPLUS 分别实现了前置自减和前置自增的实现，而额外添加的后置自减和后置自增则通过 assign_stmt 函数实现：

```

224  TreeNode* assign_stmt(void)
225  {
226      TreeNode* t = newStmtNode(AssignK);
227      if ((t) && (token == ID))
228          t->attr.name = copyString(tokenString);
229      match(ID);
230      if (token == ASSIGN)
231      {
232          match(ASSIGN);
233          if (t) t->child[0] = exp();
234      }
235      else if (token == PLUSEQ)
236      {
237          match(PLUSEQ);
238          //if (t) t->child[0] = exp();
239          if (t) {
240              TreeNode* p = newExpNode(OpK);
241              if (p != NULL) {
242                  TreeNode* temp = newExpNode(IdK);
243                  if (temp != NULL)
244                      temp->attr.name = copyString(t->attr.name);
245                  p->child[0] = temp;
246                  p->attr.op = PLUS;
247                  p->child[1] = exp();
248                  t->child[0] = p;
249              }
250          }
251      }
252      else if (token == REGEX)
253      {
254          match(REGEX);
255          if (t) t->child[0] = regex_stmt();
256      }
257      else if (token == PLUSPLUS)
258      {
259          match(PLUSPLUS);
260          if (t) {
261              TreeNode* p = newExpNode(OpK);
262              if (p != NULL) {
263                  TreeNode* temp = newExpNode(IdK);
264                  if (temp != NULL)
265                      temp->attr.name = copyString(t->attr.name);
266                  p->child[0] = temp;
267              }
268              p->attr.op = PLUS;
269              TreeNode* constNode = newExpNode(ConstK);

```

```

270         if (constNode != NULL) {
271             constNode->attr.val = 1;
272             p->child[1] = constNode;
273         }
274         t->child[0] = p;
275     }
276 }
277 else if (token == SUBSUB)
278 {
279     match(SUBSUB);
280     if(t) {
281         TreeNode* p = newExpNode(OpK);
282         if (p != NULL) {
283             TreeNode* temp = newExpNode(IdK);
284             if (temp != NULL)
285                 temp->attr.name = copyString(t->attr.name);
286             p->child[0] = temp;
287         }
288         p->attr.op = MINUS;
289         TreeNode* constNode = newExpNode(ConstK);
290         if (constNode != NULL) {
291             constNode->attr.val = 1;
292             p->child[1] = constNode;
293         }
294         t->child[0] = p;
295     }
296 }
297 return t;
298 }

```

图 4-2-10、图 4-2-11 修改后的 assign_stmt 函数

4-2-3 四元组中间代码生成的实现:

通过 Quad 数据结构，采用三地址码方式表示四元组，通过 newTemp()和 newLabel()分别产生唯一的临时变量和标签；通过 emitQ()函数实现把四元组压入名为 quads 的 vector 容器中，所有生成动作都集中于此；op2str()函数则把词法 TokenType 映射成四元式里的操作符字符串，具体如下图所示：

```

// 四元组数据结构与状态
struct Quad { QString op, a1, a2, res; };
 QVector<Quad> quads;
 int tempCnt = 0, lblCnt = 0;
 auto newTemp = [&]() { return QString("t%1").arg(++tempCnt); };
 auto newLabel= [&]() { return QString("L%1").arg(++lblCnt); };
 auto emitQ    = [&](const QString &op,
                    const QString &a1 = QString(),
                    const QString &a2 = QString(),
                    const QString &r  = QString())
 {
     quads.append({op, a1, a2, r});
 };
 auto op2str = [&](TokenType t)->QString {
     switch (t) {
     case PLUS:  return "+";
     case MINUS: return "-";
     case TIMES: return "*";
     case OVER:  return "/";
     case MOD:   return "%";
     case POWER: return "^";
     case EQ:    return "==";
     case LT:    return "<";
     case LTEQ:  return "<=";
     case RT:    return ">";
     case RTEQ:  return ">=";
     case NOTEQ: return "<>";
     default:    return "?";
     }
 };

```

图 4-2-12 四元组存储数据结构

通过函数 `genExp(TreeNode* t)` 实现对表达式的翻译，先递归左、右子树生成子表达式，在分配临时变量 `tmp`，把当前操作转变生成为一条四元组，具体如下图所示：

```

// 递归生成表达式
std::function<QString(TreeNode*)> genExp = [&](TreeNode* t)->QString {
    if (!t) return QString();
    if (t->nodekind == ExpK && t->kind.exp==OpK) {
        QString left = genExp(t->child[0]);
        QString right = genExp(t->child[1]);
        QString tmp = newTemp();
        emitQ(op2str(t->attr.op), left, right, tmp);
        return tmp;
    }
    else if (t->nodekind == ExpK && t->kind.exp==ConstK)
        return QString::number(t->attr.val);
    else if (t->nodekind == ExpK && t->kind.exp==IdK)
        return t->attr.name;
    return QString();
};

```

图 4-2-13 genExp 函数

通过 genStmt 函数，采用显式保存 sibling，再进入 switch 的方法，确保每个语句节点生产完成后才进入下一个节点，具体如下：

```
196 | // 递归生成语句序列
197 | std::function<void(TreeNode*)> genStmt = [&](TreeNode* t) {
198 |     // 用 while + 显式 next，确保每次处理一个节点
199 |     // 并且 case IfK/RepeatK 里再也不用担心 sibling 链跑偏
200 |     while (t) {
201 |         TreeNode* next = t->sibling; // 先记下一个节点
202 |
203 |         if (t->nodekind == StmtK) {
204 |             switch (t->kind.stmt) {
205 |
206 |                 case AssignK: {
207 |                     QString v = genExp(t->child[0]);
208 |                     emitQ(":", v, "", t->attr.name);
209 |                 } break;
210 |
211 |                 case ReadK:
212 |                     emitQ("read", t->attr.name);
213 |                     break;
214 |
215 |                 case WriteK: {
216 |                     QString v = genExp(t->child[0]);
217 |                     emitQ("write", v);
218 |                 } break;
219 |
220 |                 case IfK: {
221 |                     QString cond = genExp(t->child[0]); // 条件
222 |                     QString Lelse = newLabel(); // else:
223 |                     QString Lend = newLabel(); // end:
224 |
225 |                     emitQ("ifFalse", cond, "goto", Lelse); // 不满足 → else
226 |
227 |                     genStmt(t->child[1]); // then 块
228 |                     emitQ("goto", "", "", Lend); // then 结束 → end
229 |
230 |                     emitQ("label", "", "", Lelse); // else:
231 |                     if (t->child[2]) // 有无 else 都要生成标签
232 |                         genStmt(t->child[2]); // else 块
233 |
234 |                     emitQ("label", "", "", Lend); // end:
235 |                 } break;
236 |             }
```

```

237 case WhileK: {
238     QString Lbegin = newLabel(), Lbody = newLabel(), Lend = newLabel();
239     emitQ("label", "", "", Lbegin);
240     QString c = genExp(t->child[0]);
241     emitQ("ifFalse", c, "goto", Lend);
242     emitQ("label", "", "", Lbody);
243     genStmt(t->child[1]);
244     emitQ("goto", "", "", Lbegin);
245     emitQ("label", "", "", Lend);
246 } break;
247
248 case Fork: {
249     // init
250     genStmt(t->child[0]);
251     QString Lchk = newLabel(), Lbody = newLabel(), Lend = newLabel();
252     emitQ("label", "", "", Lchk);
253     {
254         QString c = genExp(t->child[1]);
255         emitQ("ifFalse", c, "goto", Lend);
256     }
257     emitQ("label", "", "", Lbody);
258     genStmt(t->child[3]); // body
259     genStmt(t->child[2]); // iter
260     emitQ("goto", "", "", Lchk);
261     emitQ("label", "", "", Lend);
262 } break;
263
264 case RepeatK: {
265     QString Lbegin = newLabel();
266     emitQ("label", "", "", Lbegin); // 入口
267
268     genStmt(t->child[0]); // 循环体
269
270     QString cond = genExp(t->child[1]); // UNTIL 后布尔表达式
271     if (cond.isEmpty()) { // 绝不能为空—否则四元组错误
272         QMessageBox::critical(nullptr, "错误", "repeat-until 条件生成失败");
273     }
274     emitQ("ifFalse", cond, "goto", Lbegin); // 条件假 → 回头
275 } break;
276
277 default:
278     // 其它不处理
279     break;
280 }

```

```

283 // 移到原本的 sibling
284 t = next;
285 }
286 };
287
288 genStmt(root);

```

图 4-2-14、图 4-2-15、图 4-2-16 genStmt 函数

最后，顺序打印带行号的四元组中间代码，行号可看作为指令地址，之后通过 QT 控件 Plain Text Edit 输出显示到屏幕上供用户查看：

```

ui->plainTextEdit_Generate->clear();
for (int i = 0; i < quads.size(); ++i) {
    const auto &q = quads[i];
    ui->plainTextEdit_Generate->appendPlainText(
        QString("%1:\t(%2, %3, %4, %5)")
            .arg(i, 2)
            .arg(q.op)
            .arg(q.a1)
            .arg(q.a2)
            .arg(q.res)
    );
}

```

图 4-2-17 输出显示用代码

四元组中间代码

```

0:      (read, x, , )
1:      (<, x, 10, t1)
2:      (ifFalse, t1, goto, L1)
3:      (+, x, 1, t2)
4:      (*, t2, 2, t3)
5:      (:=, t3, , y)
6:      (goto, , , L2)
7:      (label, , , L1)
8:      (-, x, 1, t4)
9:      (:=, t4, , y)
10:     (label, , , L2)
11:     (label, , , L3)
12:     (/ , y, 2, t5)
13:     (:=, t5, , z)
14:     (:=, z, , y)
15:     (==, y, 0, t6)
16:     (ifFalse, t6, goto, L3)
17:     (write, y, , )|

```

图 4-2-18 输出显示效果展示

5. 系统测试

5-1 系统启动

由于已经生成了 Release 版本，直接双击可执行程序即可打开：



图 5-1-1 系统运行成功截图

5-2 数据测试

针对修改与扩充后的 TINY 语言语法规则，一共有 5 组测试，其测试样例与测试结果如下图所示：

测试 1：



图 5-2-1 测试 1 成功

测试 2：



图 5-2-2 测试 2 成功

测试 3:



图 5-2-3 测试 3 成功

测试 4:



图 5-2-4 测试 4 成功

测试 5:



图 5-2-5 测试 5 成功

综上，根据测试数据中的内容进行各项测试，其结果均显示正确，结果放在测试结果文件夹中

四、实验总结（心得体会）

本次实验，我基于 QT 开发，进一步熟悉 QT 平台的使用；同时本次实验的软件基于实验三的开发而来，提高了我的效率和编程能力，为我日后工作提供了宝贵的实战经验

在本次实验中，通过对 TINY 语言语法规则的进一步了解，我深刻了解了一门编程语言设计的复杂性与健壮性，在解决对 TINY 语言原有文法理解和对其进行修改、扩容的问题过程中，我也提高了自身阅读代码、分析代码的能力

与此同时，通过实现生成四元组中间代码的功能，我加强了对四元组的理论知识的理解，并将其投入到实践当中

综上所述，本次实验锻炼了我阅读代码能力、理解代码功能能力、在新环境和条件下复现程序功能的能力以及在时间紧张情况下完成任务的抗压能力，加深了我对编程语言设计、软件开发的理解，获益良多

五、参考文献：

编译原理（3）——for 循环_编译原理实验三语法分析 for 循环-CSDN 博客：

<https://blog.csdn.net/zsh1184528359/article/details/109635198>